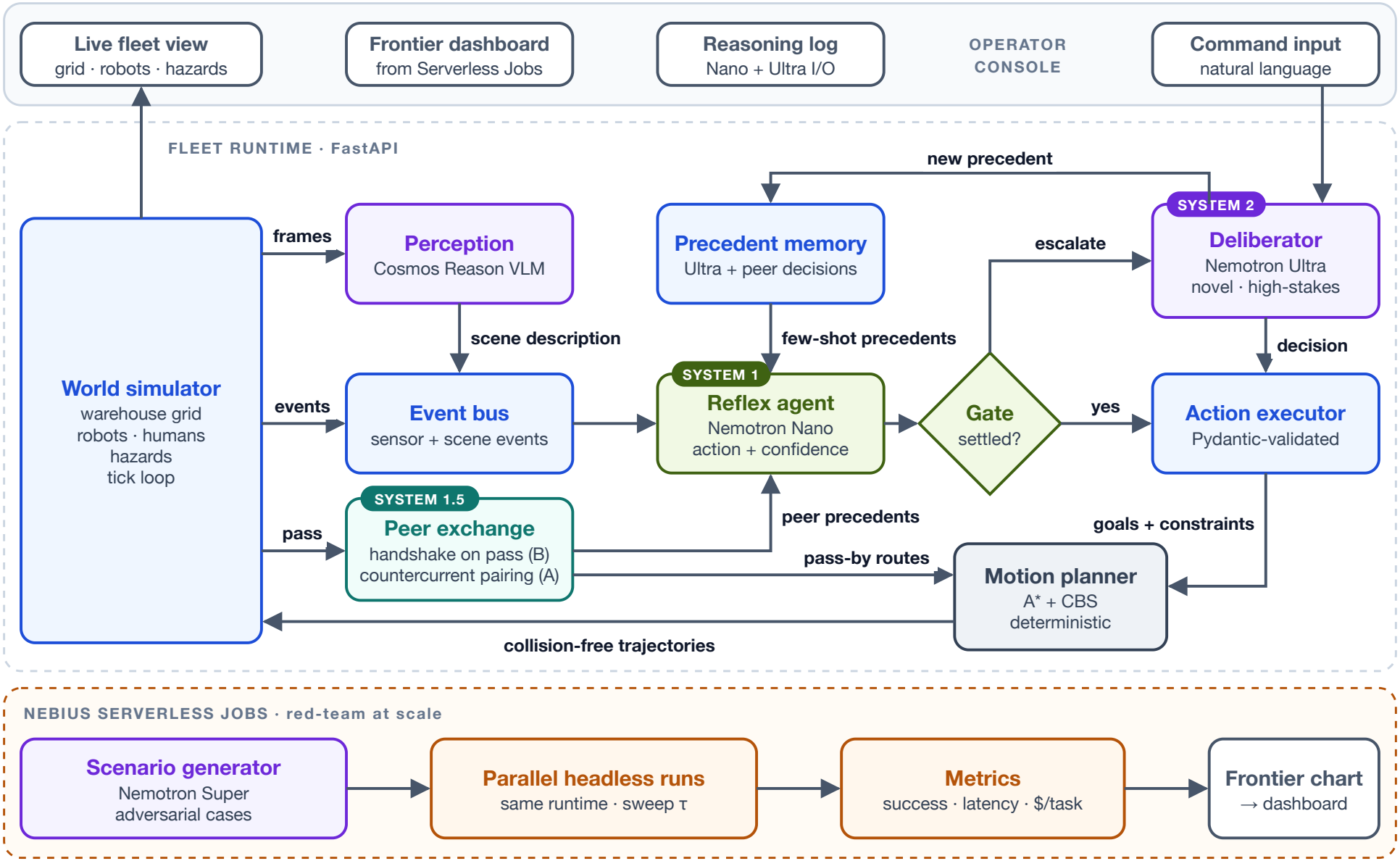
NemoFleet · System 1 / System 2 for robot fleets

SYSTEM MAP

A fast model on each robot handles routine events, peers share what the fleet already knows when they pass, and a large reasoning model in the cloud handles only what is novel or high-stakes. A deterministic planner owns every trajectory.

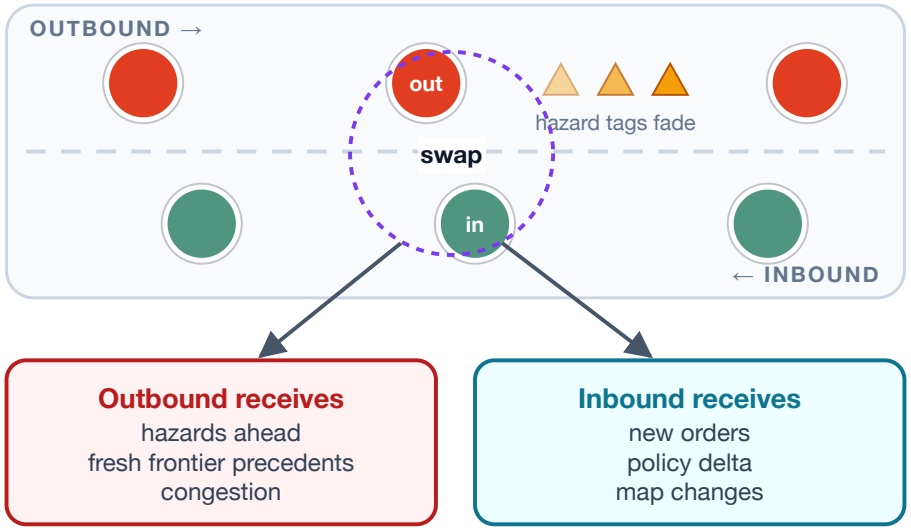


Fleet runtime · System 1 · reflex · System 1.5 · peers · NVIDIA models (Token Factory) · Deterministic planning · Serverless Jobs

Robots teach each other in passing

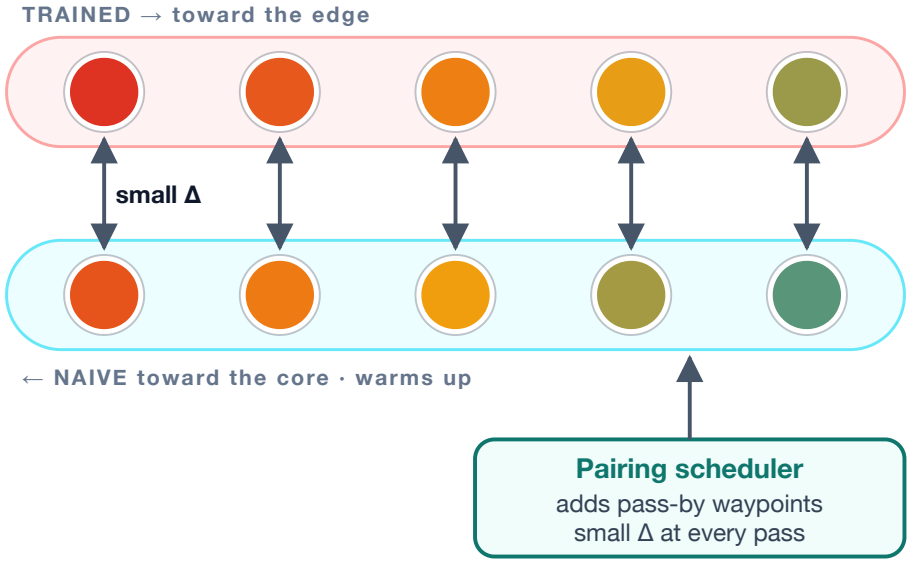
Two designs from the countercurrent options work together. **B** moves knowledge at runtime: robots swap a small payload whenever they pass. **A** shapes who passes whom, so each robot meets peers only slightly ahead of it. Most already-seen problems get settled without a cloud call.

B · Handshake on pass (runtime)



No model call · bounded payload · Cosmos called once per hazard

A · Countercurrent pairing (learning)



Peers never override safety
A human present, an unknown hazard class or a blocked exit always escalates to Ultra.

Provenance on every precedent
Who decided it (Ultra or a peer), when, and how many hops. Confidence decays per hop.

Everything expires
Precedents and hazard tags carry a TTL, so a bad lesson can't circulate forever.

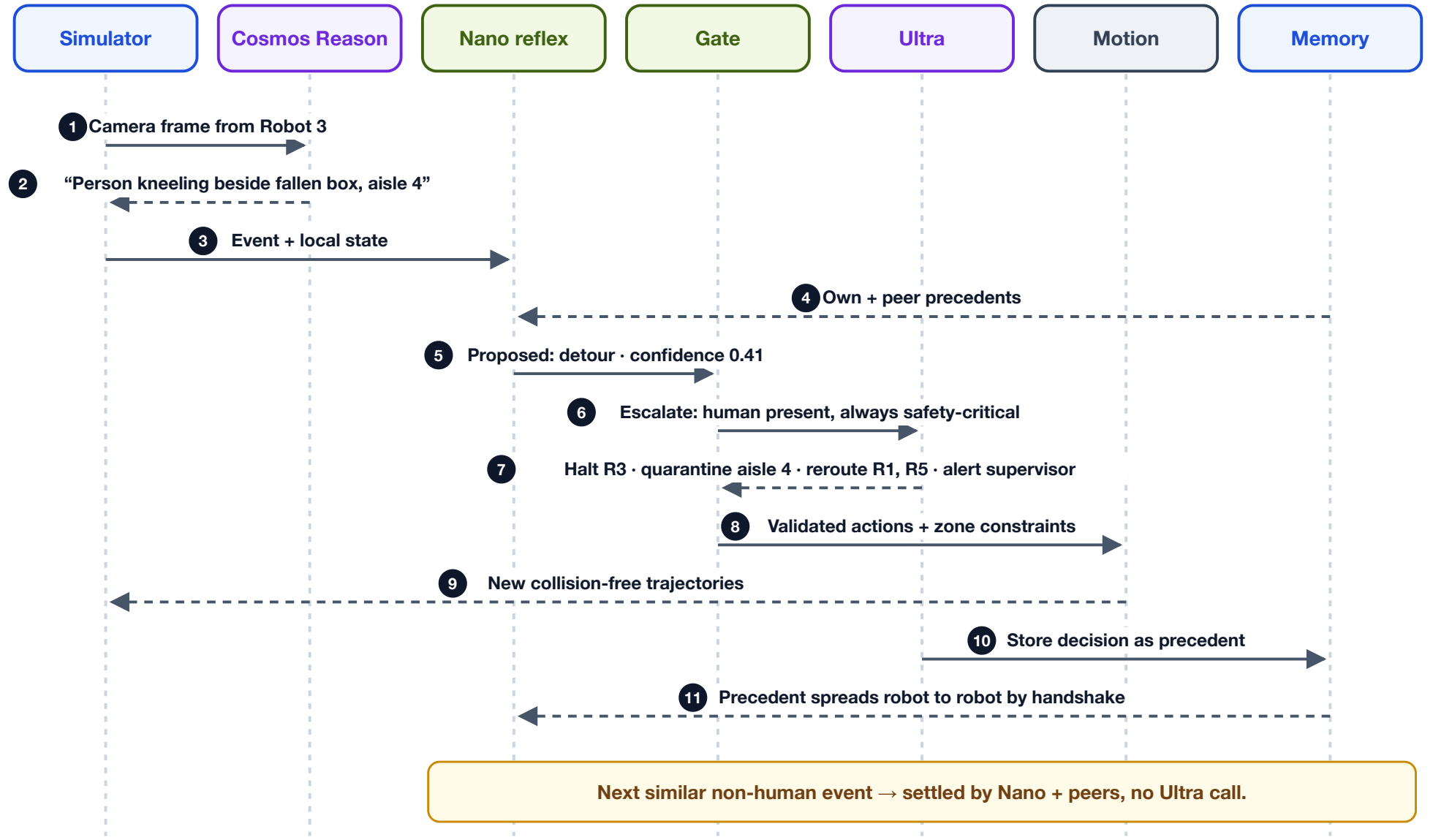
Conflicts escalate
Ultra outranks peers. Two precedents that disagree are themselves an escalation trigger.

Fleet runtime · System 1 · reflex · System 1.5 · peers · NVIDIA models (Token Factory) · Deterministic planning · Serverless Jobs

A person enters a robot lane

ESCALATION SEQUENCE

Humans are always safety-critical, so this escalates to Ultra; the decision then spreads robot to robot by handshake.



Fleet runtime · System 1 · reflex · System 1.5 · peers · NVIDIA models (Token Factory) · Deterministic planning · Serverless Jobs

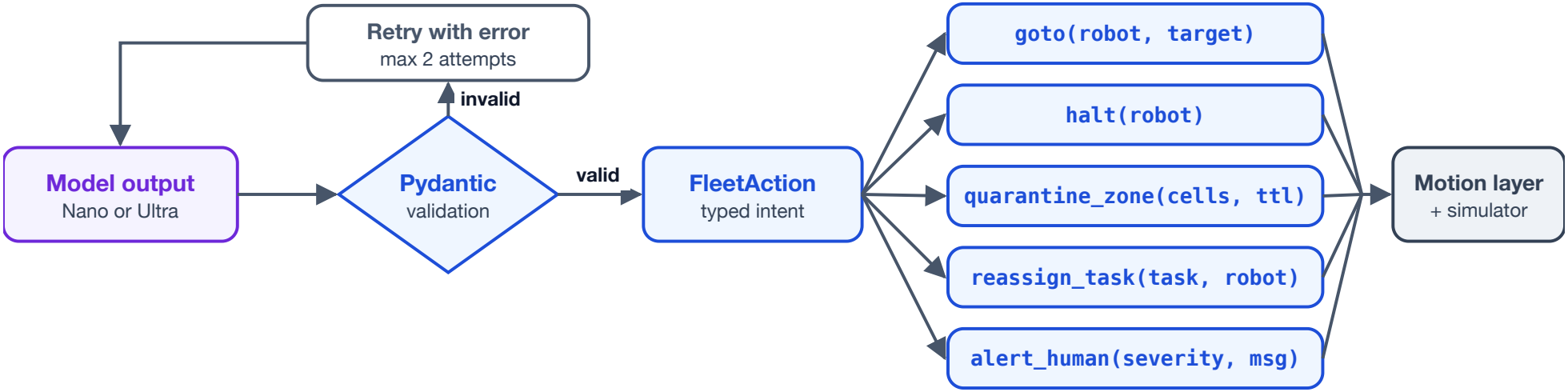
- - - dashed arrow = response / return

Who decides what

> **Principle:** models decide *what* should happen; the planner decides *how* to move. Every model output must pass schema validation before anything moves.

LAYER	MODEL / ALGORITHM	DECIDES	LATENCY	NEVER DOES
Perception	Cosmos Reason	What the camera sees, on first sighting	~1 s	Plan actions
Reflex	Nemotron Nano	Routine responses + a confidence score	< 1 s	Handle safety-critical novel events alone
Peer exchange	Handshake + fading tags + pairing	What the fleet already knows reaches this robot	< 50 ms	Override a safety escalation
Escalation gate	Rules + threshold τ	Settled locally, or System 2 needed	~0	Generate content
Deliberation	Nemotron Ultra	Novel, high-stakes decisions; task re-prioritization	5–30 s	Produce raw coordinates
Motion	A* + CBS	Collision-free paths + pass-by waypoints	< 10 ms	Interpret semantics
Executor	Pydantic schemas	Whether an action is valid	~0	Accept malformed LLM output

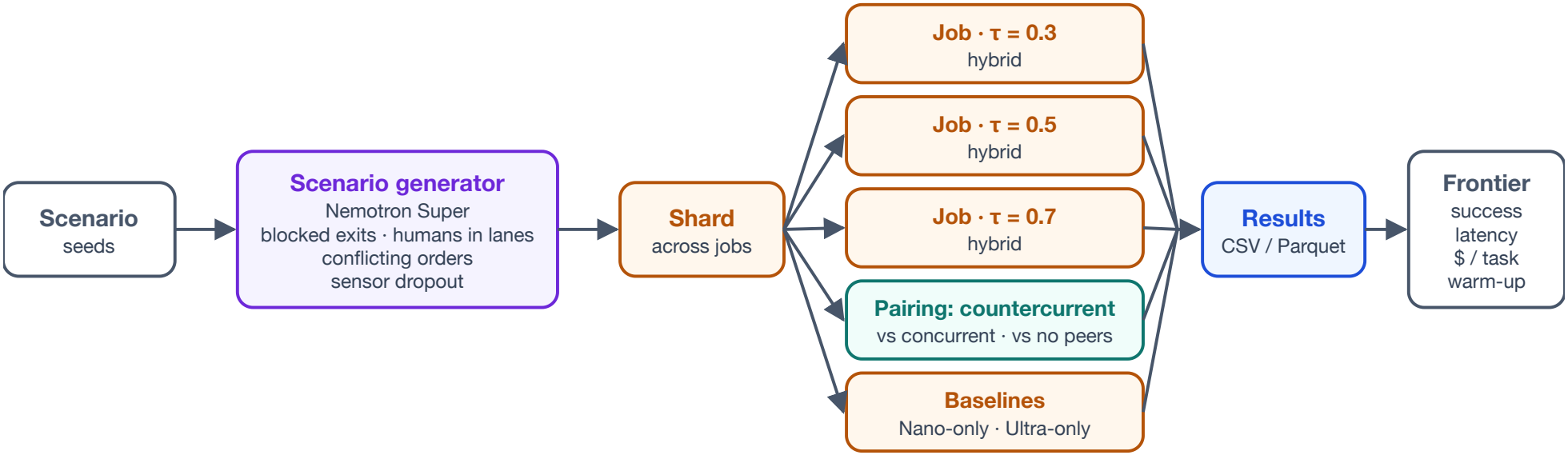
Action schema boundary



Fleet runtime · System 1 · reflex · System 1.5 · peers · NVIDIA models (Token Factory) · Deterministic planning · Serverless Jobs

Measuring when to escalate

Serverless Jobs run adversarial scenarios in parallel across escalation thresholds τ and pairing strategies: the escalation frontier, plus countercurrent vs. concurrent vs. no-peer warm-up and hazard-spread curves.



Where NVIDIA and Nebius are used

COMPONENT	NVIDIA MODEL	NEBIUS SERVICE
Reflex agent	Nemotron Nano	Token Factory
Deliberator	Nemotron Ultra	Token Factory
Perception	Cosmos Reason	Token Factory or AI Cloud endpoint
Scenario generator	Nemotron Super	Token Factory
Peer exchange	None: no model call is the point	—
Red-team + pairing sweeps	—	Serverless Jobs

Model IDs to be confirmed against the Token Factory catalog.

Fleet runtime · System 1 · reflex · System 1.5 · peers · NVIDIA models (Token Factory) · Deterministic planning · Serverless Jobs

Code map: nemofleet/

PLANNED MODULE LAYOUT

models.py

WorldState, Robot, Event
FleetAction, Decision, Precedent

sim/

world.py — grid, robots, humans, tick loop
proximity.py — passes, handshakes, fading tags
scenarios.py — loading + generation

motion/

astar.py — single-agent paths
cbs.py — multi-agent conflict search

cognition/

client.py — Token Factory client
reflex.py — Nano · System 1
peers.py — payloads, provenance, TTL (B)
pairing.py — countercurrent scheduler (A)
deliberator.py — Ultra · System 2
gate.py — escalation + safety overrides
perception.py — Cosmos Reason
memory.py — precedent store

api/

server.py — /state /command /inject /tick /log

jobs/

redteam.py — τ sweep + pairing sweep
frontier.py — aggregation + charts

ui/

Operator console: live view, command input, reasoning log, frontier dashboard

Fleet runtime · System 1 · reflex · System 1.5 · peers · NVIDIA models (Token Factory) · Deterministic planning · Serverless Jobs